Data Leakage

Data leakage is a problem in machine learning where information from outside the training dataset improperly influences the model, leading to overly optimistic performance estimates. It occurs when data that would not be available at prediction time is used during model training, compromising the model's ability to generalize.

Key Facts

- Type: Machine learning training issue
- Main risk: Inflated model accuracy during validation
- Root cause: Information leakage from test or future data
- Impact: Poor real-world performance and unreliable metrics

Causes and Mechanisms

Data leakage commonly arises when preprocessing steps are applied before data splitting, causing the training process to incorporate knowledge from the validation or test sets. It can also occur if target variables, future data, or correlated proxy features are unintentionally included during training. Leakage often hides within feature engineering or time-based data handling errors.

Detection and Prevention

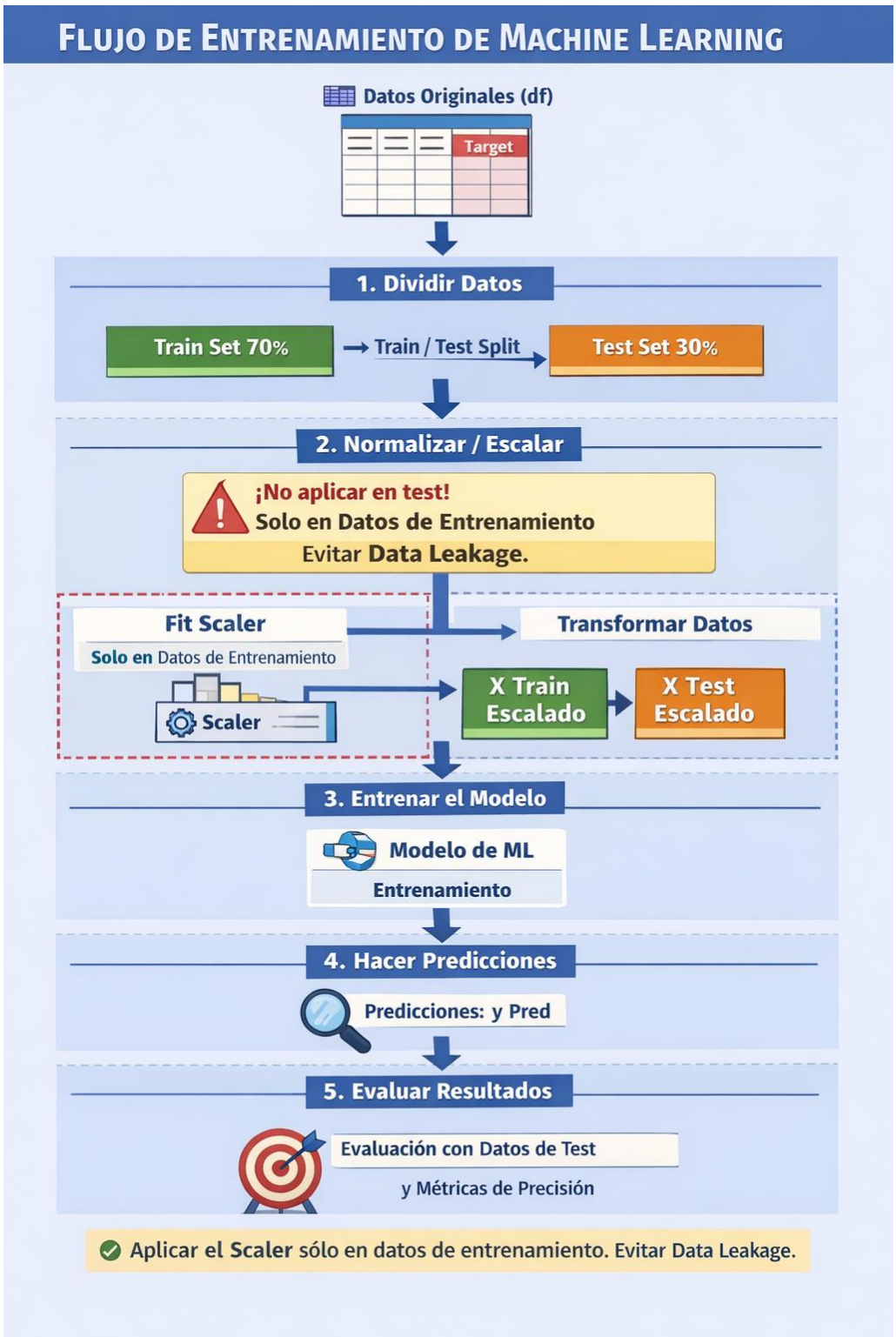
Preventing leakage requires rigorous data management practices. Splitting data into training, validation, and test sets should occur before any transformations or scaling. Features derived from future information must be excluded, and time-aware validation strategies should be used for sequential data. Periodic audits and pipeline reviews help detect hidden leakage.

Impact on Model Performance

Models affected by data leakage tend to show unrealistically high accuracy, precision, or recall during development. However, when deployed, their performance often drops sharply because the real-world inputs lack the leaked information the model relied on. This mismatch leads to mistrust in predictive analytics and poor business decisions.

Common Examples

- Normalizing the entire dataset before splitting.
- Including post-event features (e.g., outcomes) in training data.
- Leaking labels through data imputation or encoding.
- Sharing cross-validation folds incorrectly in time-series tasks.



Código Explicado

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X = df.drop('target', axis=1)
y = df['target']

# 1. Split data into training and testing sets FIRST
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 2. Initialize the scaler
scaler = StandardScaler()

# 3. Fit the scaler ONLY on the training data and transform it
X_train_scaled = scaler.fit_transform(X_train)

# 4. Transform the test data using the fitted scaler (DO NOT fit on test data)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=5000)
# 5. Train the model using the scaled training data
model.fit(X_train_scaled, y_train)

# 6. Make predictions using the scaled test data
y_pred = model.predict(X_test_scaled)
accuracy_score(y_test, y_pred)
```

1. Importación de librerías

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Módulo	Función
train_test_split	Divide datos en entrenamiento y prueba
StandardScaler	Escala variables numéricas
LogisticRegression	Modelo de clasificación
accuracy_score	Métrica de evaluación

2. Separar variables predictoras y target

```python
X = df.drop('target', axis=1)
y = df['target']
```

Qué ocurre aquí

- X → variables independientes (*features*)

- y → variable objetivo (*label*)
  Ejemplo:

edad	ingreso	compra
25	500	0
40	2000	1

X = edad + ingreso
y = compra

3. **Dividir datos (PASO CRÍTICO)**

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Qué hace
Divide el dataset:
- **70% entrenamiento**
- **30% prueba**

```
Dataset original
       ↓
 ┌──────────────┬──────────────┐
 │ Train (70%)  │ Test (30%)   │
 └──────────────┴──────────────┘
```

Por qué se hace primero

Para simular datos **nunca vistos**.

Si escalas antes de dividir → el modelo "ve el futuro".
Eso genera **Data Leakage**.
Desde este momento:
- **TRAIN** → datos para aprender
- **TEST** → simulación del futuro
  El test representa datos reales que aún no existen.

4. **Crear el escalador**

```
scaler = StandardScaler()
```

Aquí solo definimos el objeto.
Aún NO aprende nada.

5. **Ajustar el scaler SOLO con entrenamiento**

```
X_train_scaled = scaler.fit_transform(X_train)
```

Este paso tiene dos operaciones:

a) **fit**

Calcula estadísticas del training:

$$z = \frac{x - \mu}{\sigma}$$

Aprende:

- media (μ)
- desviación estándar (σ)

b) **transform**

Aplica la normalización.

Ejemplo:

Variable	Media	Std
Edad	35	10

Edad 45 → (45−35)/10 = 1

Ahora todas las variables quedan comparables.

6. **Escalar datos de prueba (SIN aprender)**

```
X_test_scaled = scaler.transform(X_test)
```

Punto más importante del código.
NO usamos fit_transform.
Solo: **transform()**

Porque:

- el modelo solo puede aprender del entrenamiento
- el test debe permanecer desconocido

7. **Crear modelo de clasificación**

```
model = LogisticRegression(max_iter=5000)
```

Se instancia el modelo de Logistic Regression.
Características:

- Clasificación binaria o multiclase
- Predice probabilidades
- Función sigmoide

max_iter=5000
aumenta iteraciones para asegurar convergencia.

8. **Entrenar el modelo**

```
model.fit(X_train_scaled, y_train)
```

El modelo aprende:
Features escaladas → relación → target

Internamente:

- optimiza coeficientes
- minimiza función de pérdida
- encuentra frontera de decisión

9. **Realizar predicciones**

```
y_pred = model.predict(X_test_scaled)
```

Ahora el modelo:

- recibe datos **nunca vistos**
- genera clases predichas.

10. **Evaluar desempeño**

```
accuracy_score(y_test, y_pred)
```

Calcula:

$$Accuracy = \frac{predicciones\ correctas}{total}$$

Ejemplo:

- 90 correctas / 100 muestras
- Accuracy = **0.90**

Pipeline completo (visión profesional)

```
Datos crudos
      ↓
Split (Train/Test)
      ↓
Fit Scaler (solo Train)
      ↓
Transform Train + Test
      ↓
Entrenar Modelo
      ↓
Predecir
      ↓
Evaluar
```

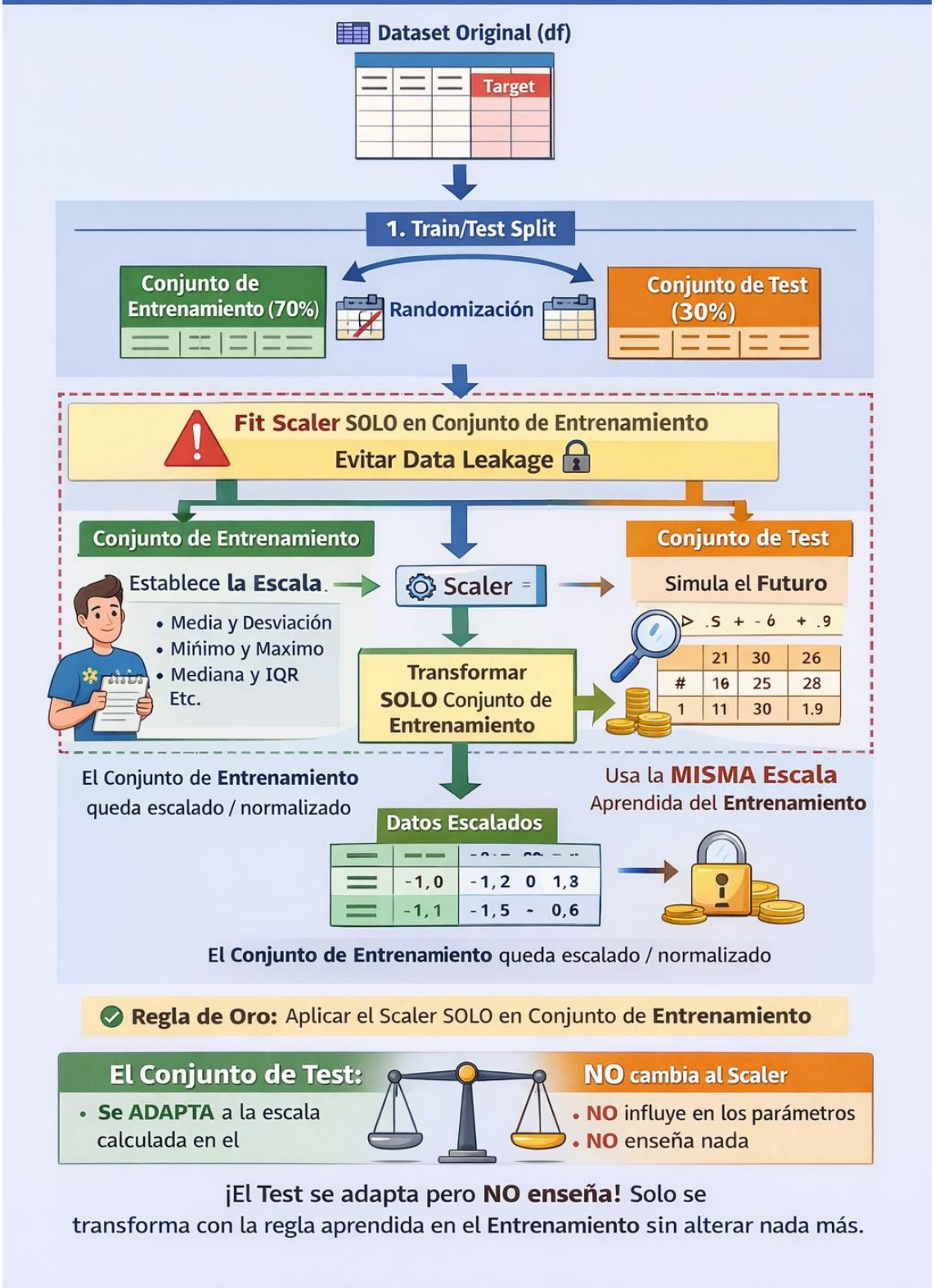
Insight clave

1. **NO aprende nada nuevo**
2. Usa la media y desviación del TRAIN
3. Aplica la misma fórmula

   X_test queda en **la misma escala estadística** que X_train.

4. TRAIN aprende la escala
5. TEST solo se adapta a ella
6. NO se recalcula la media del test
7. Cuando normalizas o escalas, el objetivo es:
   a. poner TRAIN y TEST en la misma escala matemática
   b. sin permitir que el TEST influya en el aprendizaje

¿Qué sucede al Normalizar/Escalar el Conjunto de TEST?



Regla de Oro: Aplicar el Scaler SOLO en Conjunto de **Entrenamiento**

El Conjunto de Test:

- **Se ADAPTA** a la escala calculada en el

NO cambia al Scaler

- **NO** influye en los parámetros
- **NO** enseña nada

¡El Test se adapta pero **NO enseña!** Solo se transforma con la regla aprendida en el **Entrenamiento** sin alterar nada más.